

El chip que nadie te explicó

Cómo Windows, macOS, Linux y Android ya le dieron a la IA las llaves de tu hardware

Sistemas Operativos — Facultad de Ingeniería, UNAM

Carballido Carrillo Carlos Fernando y Sánchez Soperanes Raúl H.

El dato inicial: el número que exige una “PC con IA”

Microsoft exige que una computadora logre al menos 40 billones de operaciones por segundo (40 TOPS) solo para poder llevar la etiqueta “Copilot+ PC”, su sello oficial de “PC con IA” (Microsoft, s.f.-b). Si el equipo no alcanza ese número, no puede usar el nombre, sin importar qué tan bien funcione su inteligencia artificial en la práctica. Esa cifra es el punto de partida de esta investigación: detrás de cada función que las empresas llaman “inteligencia artificial” hay una decisión de sistema operativo sobre qué chip usa, cuándo y por qué —y, a veces, también hay una decisión de mercadotecnia.

¿Qué investigamos?

En este trabajo investigamos cómo los cuatro sistemas operativos más usados del mundo —Windows, macOS, Linux y Android— administran la inteligencia artificial como lo que realmente es a nivel técnico: un recurso de hardware más, igual que la memoria o el procesador. No profundizamos en los algoritmos de aprendizaje automático en sí mismos, sino en la pregunta que sí es, en el fondo, sistemas operativos: ¿quién decide qué chip ejecuta cada tarea de IA, y bajo qué criterios? La investigación se organiza en cuatro bloques: primero, el concepto de NPU y por qué administrarla es un problema de sistemas operativos; segundo, cuatro casos reales —Android, Windows, macOS y Linux—, cada uno con su propio dato sorprendente; tercero, una comparación directa entre los cuatro sistemas; y cuarto, una reflexión crítica sobre qué tanto de esta tendencia es necesidad técnica real y qué tanto es mercadotecnia.

Introducción

Hoy es común escuchar que cualquier función de una computadora o un teléfono “usa inteligencia artificial”, pero pocas veces se explica qué significa eso a nivel de sistema operativo. Este trabajo aborda el tema desde esa perspectiva: no como algoritmos de aprendizaje automático, sino como un problema de administración de recursos, que es la función central de cualquier sistema operativo. La idea de que el sistema operativo deba gestionar un acelerador de hardware especializado no es nueva ni es solo una moda reciente: ya en 2012, investigadores de la Universidad de Wisconsin-Madison argumentaban formalmente que los sistemas operativos debían asumir la administración de aceleradores de hardware, en vez de tratarlos como dispositivos genéricos sin supervisión (Panneerselvam y Swift, 2012). Dos años después, un grupo de la Universidad de Rochester publicó en ASPLOS —una de las conferencias más importantes en sistemas operativos— el diseño de un esquema de planificación justo para acceso a aceleradores de cómputo rápidos, aplicado a GPUs (Menychtas, Shen y Scott, 2014). La NPU (Unidad de Procesamiento Neuronal) que hoy traen los equipos nuevos es, en esencia, el acelerador más reciente de esa misma línea de investigación, y el problema de administrarlo sigue siendo, en el fondo, el mismo.

¿Qué es una NPU y por qué es un problema de sistemas operativos?

Un sistema operativo administra los recursos de hardware: decide qué proceso usa el procesador, cómo se reparte la memoria y qué dispositivo atiende cada tarea. La NPU es un chip diseñado específicamente para los cálculos de redes neuronales, más rápido y con mucho menor consumo de batería que la CPU o la GPU para ese tipo de operación (Microsoft, s.f.-a). Al ser un recurso nuevo, el sistema operativo tiene que decidir, para cada tarea de IA, si la envía a la CPU, a la GPU o a la NPU —el mismo tipo de decisión que ya toma al planificar procesos o dispositivos de entrada/salida, solo que ahora el recurso en disputa es un acelerador especializado. La investigación académica reciente confirma que este sigue siendo un problema abierto: un estudio de 2025 señala que el planificador que traen las NPU de fábrica no considera requerimientos de tiempo real, y propone un modelo propio para predecir cuánto tardará una tarea cuando corre junto a otras en la misma NPU (Yao et al., 2025). Esto confirma que administrar una NPU no es un problema resuelto ni trivial, sino un tema

activo de investigación en sistemas operativos.

Android: fotografía computacional y batería adaptable

Fotografía computacional (HDR+ y Night Sight)

Al tomar una foto en un Android moderno, la cámara captura varias imágenes en sucesión rápida. El sistema operativo coordina la cámara, el procesador de imagen (ISP) y un modelo de IA que combina las capturas en una sola fotografía con mejor exposición y menos ruido (Google Research, 2014). La misma idea aplicada a poca luz da lugar a Night Sight. En los Pixel recientes, este procesamiento ocurre en el propio chip del teléfono, sin conexión a internet (Google, 2021).

Gestión de batería (Adaptive Battery)

Google, junto con DeepMind, desarrolló “Adaptive Battery”: en vez de una regla fija, el sistema aprende qué apps usa más el usuario y les da prioridad de recursos, limitando en segundo plano a las demás (Android Developers Blog, 2018). Es el mismo problema de administración de procesos visto en clase, solo que la decisión la toma un modelo entrenado en vez de una política estática como round robin.

Windows: Auto Super Resolution y Recall

Los equipos “Copilot+ PC” exigen una NPU de al menos 40 billones de operaciones por segundo (Microsoft, s.f.-b). Auto Super Resolution usa ese chip para reconstruir, con IA, los detalles de un videojuego renderizado a menor resolución —el juego no necesita modificarse, es el propio sistema operativo quien intercepta y mejora la imagen (Microsoft, s.f.-c). Recall, por su parte, toma capturas de pantalla periódicas y las analiza con varios modelos pequeños en la NPU para permitir búsquedas en lenguaje natural sobre el historial visual del usuario, todo almacenado localmente y cifrado (Microsoft, s.f.-d). Esta función generó controversia real porque también podía capturar contraseñas visibles en pantalla, lo que obligó a reforzar sus controles de privacidad (Microsoft, s.f.-e) —un buen ejemplo de que integrar IA en el sistema operativo trae beneficios, pero también riesgos que deben evaluarse.

macOS: Apple Intelligence y el Neural Engine

Los Mac con Apple Silicon incluyen un “Neural Engine”. Apple Intelligence lo usa para tareas como corregir texto o resumir correos directamente en el dispositivo, sin enviar información a internet en la mayoría de los casos (Apple, s.f.). Cuando una solicitud es demasiado compleja para el modelo local, macOS la envía a “Private Cloud Compute”, un servidor diseñado para no almacenar los datos del usuario (Apple, s.f.). Es otro caso claro de cómo el sistema operativo decide automáticamente dónde ejecutar cada tarea: localmente o en un servidor especializado.

Linux: una filosofía distinta

A diferencia de Windows y macOS, Linux no trae un asistente de IA integrado por defecto. Sí existe soporte técnico real: el kernel incluye controladores de código abierto para NPUs de distintos fabricantes, como AMD, dentro de un subsistema dedicado a aceleradores (The Linux Kernel Organization, s.f.). La diferencia es de filosofía: usar o no estas funciones queda en manos del usuario, y Canonical (la empresa detrás de Ubuntu) ha declarado públicamente su intención de ofrecer IA local y opcional, sin convertir el sistema operativo en lo que ellos mismos llaman “un truco publicitario de IA” (Larabel, 2026).

Un caso técnico más a fondo: sched_ext en Linux

Para entender hasta dónde puede llegar esta integración, vale la pena ver un caso concreto dentro del propio planificador de procesos de Linux, el componente que decide qué proceso usa el procesador en cada instante. Durante años, Linux usó un solo planificador general para todo: primero CFS y, desde la versión 6.6 (2023), EEVDF. Para probar una política distinta había que modificar y recompilar el kernel completo, algo lento incluso para investigadores. Eso cambió en la versión 6.12 del kernel (2024), con la llegada de sched_ext: una forma de cargar planificadores personalizados como programas independientes, sin tocar el kernel ni reiniciar el equipo, con una red de seguridad que regresa todo a EEVDF automáticamente si el programa cargado falla.

Esto abrió la puerta a experimentar con planificadores hechos a la medida. Uno de los casos más llamativos es `scx_lavd`, desarrollado por Igalia para Valve con el objetivo de que los videojuegos respondieran mejor en la consola portátil Steam Deck. En la Linux Plumbers Conference de 2025, ingenieros de Meta reportaron que terminaron adoptando ese mismo planificador como opción en sus servidores de producción, porque reparte bien la carga entre núcleos que comparten memoria caché. Es un ejemplo real de cómo una solución pensada para un problema pequeño (que un juego no se trabe) terminó resolviendo un problema de cómputo a gran escala.

Sobre dónde entra la inteligencia artificial en todo esto, la investigación reciente (como SchedCP, un trabajo de 2025) muestra que no conviene que un modelo decida en cada instante qué tarea ejecutar, porque el planificador toma decisiones miles de veces por segundo y una inferencia de IA es demasiado lenta para ese ritmo. Lo que sí funciona es que un agente de inteligencia artificial observe el comportamiento del sistema de vez en cuando y genere o ajuste la política completa, que después corre sola a velocidad normal. En las pruebas de ese trabajo, el planificador elegido por el agente llegó a ser hasta 1.79 veces más rápido que EEVDF compilando el kernel, aunque ese es el mejor caso reportado y no un resultado garantizado en cualquier escenario.

Comparación entre los cuatro sistemas

	Windows	macOS	Linux	Android
¿IA integrada por defecto?	Sí	Sí	No	Sí
Chip dedicado	NPU	Neural Engine	NPU (opcional)	NPU / Tensor
Ejemplo destacado	Recall / Auto SR	Apple Intelligence	Soporte de controladores	HDR+ / Adaptive Battery
Filosofía	IA como parte central del producto	Privacidad + local	Opcional, decide el usuario	Mejorar la experiencia diaria

Doce años de diferencia: la línea de tiempo detrás del hype

Vale la pena situar estos hechos en una línea de tiempo. La primera propuesta académica formal para que un sistema operativo administre aceleradores de hardware es de 2012 (Panneerselvam y Swift, 2012); el primer esquema concreto de planificación justa para acceder a esos aceleradores es de 2014 (Menychtas et al., 2014); y la investigación más reciente sobre cómo planificar tareas específicamente dentro de una NPU es de apenas 2025 (Yao et al., 2025). Microsoft, en cambio, presentó el término comercial “Copilot+ PC” y el requisito de 40 TOPS hasta 2024. Es decir: la pregunta académica de “cómo administrar un acelerador” lleva doce años de investigación seria antes de que una empresa la convirtiera en una etiqueta de marketing para vender hardware nuevo. Esa diferencia de doce años es, quizás, el dato más revelador de todo este trabajo: la industria no inventó un problema nuevo, empaquetó un problema viejo con un nombre nuevo.

¿Es marketing llamarle “inteligencia artificial” a todo esto?

Vale la pena una pausa crítica. Varias de estas funciones existían, en forma más simple, antes de llamarse “inteligencia artificial”: los sistemas operativos ya administraban batería, memoria y dispositivos mucho antes de que existieran las NPU. La literatura académica respalda que administrar un acelerador de hardware es un problema técnico legítimo y de larga data en sistemas operativos (Panneerselvam y Swift, 2012; Menychtas et al., 2014), no una invención reciente de marketing. Sin embargo, también hay evidencia de uso comercial: Microsoft exige explícitamente una NPU de 40 TOPS para que un equipo se llame “Copilot+ PC” (Microsoft, s.f.-b), lo cual empuja a comprar hardware nuevo. El caso de Linux refuerza esta lectura: al no depender de vender hardware, trata el soporte de NPU como una opción técnica más, no como argumento central de venta (Larabel, 2026). La conclusión más honesta es que ambas cosas conviven: existe un problema real de administración de recursos, documentado académicamente desde hace más de una década, pero también una capa de mercadotecnia que exagera su novedad para justificar la compra de hardware nuevo.

Conclusión

Vista desde un sistema operativo, la inteligencia artificial no es una entidad que “piensa”: es un recurso de hardware que debe administrarse, repartirse y protegerse,

igual que la memoria, el procesador o los dispositivos de entrada y salida. Analizar estos casos en Android, Windows, macOS y Linux muestra que los conceptos vistos en la materia —administración de procesos, de memoria y de dispositivos— siguen vigentes frente a la tecnología más reciente, y que tanto la investigación académica como una mirada crítica permiten distinguir entre la necesidad técnica real y la estrategia comercial que la acompaña.

Referencias

Android Developers Blog. (2018). A smarter smartphone, with machine learning at the core. Google.

<https://android-developers.googleblog.com/2018/05/whats-new-in-android-p-beta.html>

Apple. (s.f.). Apple Intelligence and privacy on Mac. Soporte técnico de Apple.

Recuperado en 2026, de <https://support.apple.com/guide/mac-help/apple-intelligence-and-privacy-mchlfcd4779/mac>

Google. (2021). Pixel 6's camera combines hardware, software and ML. The Keyword.

<https://blog.google/products/pixel/pixel-6s-camera-combines-hardware-software-and-ml/>

Google Research. (2014). HDR+: Low light and high dynamic range photography in

the Google Camera app. <https://research.google/blog/hdr-low-light-and-high-dynamic-range-photography-in-the-google-camera-app/>

Larabel, M. (2026). Linux desktops embrace AI on their own terms: Local power

without the corporate overreach. WebProNews. <https://www.webpronews.com/linux-desktops-embrace-ai-on-their-own-terms-local-power-without-the-corporate-overreach/>

Menychtas, K., Shen, K., & Scott, M. L. (2014). Disengaged scheduling for fair,

protected access to fast computational accelerators. En Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14) (pp. 301–316). ACM.

<https://doi.org/10.1145/2541940.2541963>

Microsoft. (s.f.-a). How the NPU is paving the way toward a more intelligent Windows.

Microsoft Source. Recuperado en 2026, de <https://news.microsoft.com/source/features/ai/how-the-npu-is-paving-the-way-toward-a-more-intelligent-windows/>

Microsoft. (s.f.-b). Develop AI applications for Copilot+ PCs. Microsoft Learn.

Recuperado en 2026, de <https://learn.microsoft.com/windows/ai/npu-devices/>

Microsoft. (s.f.-c). What is Windows ML? Microsoft Learn. Recuperado en 2026, de <https://learn.microsoft.com/windows/ai/new-windows-ml/overview>

Microsoft. (s.f.-d). Administrar Recall. Microsoft Learn. Recuperado en 2026, de <https://learn.microsoft.com/windows/client-management/manage-recall>

Microsoft. (s.f.-e). Usar Recall para encontrar y recordar lo que has hecho en tu PC. Soporte técnico de Microsoft. Recuperado en 2026, de <https://support.microsoft.com/topic/d404f672-7647-41e5-886c-a3c59680af15>

Panneerselvam, S., & Swift, M. M. (2012). Operating systems should manage accelerators. En Proceedings of the 4th USENIX Workshop on Hot Topics in Parallelism (HotPar '12). USENIX Association. <https://www.usenix.org/conference/hotpar12/workshop-program/presentation/panneerselvam>

The Linux Kernel Organization. (s.f.). accel/amdxdna NPU driver. The Linux Kernel documentation (versión 6.17). Recuperado en 2026, de <https://www.kernel.org/doc/html/v6.17/accel/amdxdna/index.html>

Zheng, Y., & Hu, Y. (2025). Towards agentic OS: An LLM agent framework for Linux schedulers (arXiv:2509.01245). University of California, Santa Cruz. <https://arxiv.org/abs/2509.01245>

Yao, Y., Hu, Y., Dang, Y., Tao, W., Hu, K., Huang, Q., Peng, Z., Yang, G., & Zhou, X. (2025). Workload-aware performance model based soft preemptive real-time scheduling for neural processing units. IEEE Transactions on Parallel and Distributed Systems. <https://doi.org/10.1109/TPDS.2025.3553922>